

Migración de SecureNet a Cloud con Terraform

Descripción General

Este documento describe el proceso de migración del proyecto **SecureNet** desde un entorno local hacia infraestructura cloud en **AWS**, utilizando **Terraform** para la gestión de infraestructura como código (IaC).

Contexto y Evolución

Fase Actual: Laboratorios (Pruebas)

- **Entorno:** Cuenta de laboratorios de AWS
- **Propósito:** Testing y validación de la arquitectura
- **Alcance:** Pruebas de conceptos, configuración inicial, validación de procesos

Fase Objetivo: Producción (Official AWS Account)

- **Entorno:** Cuenta oficial de AWS
 - **Propósito:** Despliegue en producción
 - **Alcance:** Arquitectura estable y completa
-

Decisiones Arquitectónicas

Stack Seleccionado: AWS + Terraform

Por qué AWS:

- Estabilidad y madurez en servicios
- Amplia documentación y comunidad
- Integración con herramientas DevOps

Por qué Terraform:

- Infrastructure as Code (IaC) – versionable, reproducible
- Multi-cloud (facilita futuras migraciones)
- Gestión declarativa del estado

Alternativa Descartada: Oracle + AWS

- **Razón de descarte:** Las cuentas free tier de Oracle Cloud están saturadas y resultan imposibles de usar
 - **Impacto:** Simplifica la arquitectura a un único proveedor (AWS)
-

Arquitectura de Infraestructura

Componentes Principales

Se despliegan **2 máquinas virtuales (EC2)** especializadas:

VM 1: Jenkins (Orquestación y CI/CD)

EC2 instance: Jenkins

S0: Amazon Linux 2023
Instancia: t3.medium
Puerto 8080: Web UI de Jenkins
Puerto 50000: Agentes remotos

Servicios:

- Docker (contenedor de Jenkins)
- Docker Compose (orquestación)
- Git (control de versiones)

Responsabilidades:

- Orquestación de pipelines CI/CD
- Gestión y despliegue de aplicaciones
- Punto central de control y automatización

VM 2: Aplicaciones Principales

EC2 instance: Applications

S0: Amazon Linux 2023
Instancia: t3.medium
(a completar según servicios)

Servicios:

- Docker (contenedores de apps)
- Docker Compose (orquestación)
- Git
- Otros microservicios

Responsabilidades:

- Ejecución de aplicaciones core
- Servicios de backend
- Bases de datos y almacenamiento de datos

Componentes de Terraform

Estructura del Proyecto

```
terraform/
├── aws_compute.tf      # Instancias EC2, Security Groups
├── aws_storage.tf      # Volúmenes, snapshots (futuro)
├── oci_compute.tf      # Configuración OCI (descartada)
├── providers.tf        # Configuración de providers
├── variable.tf          # Variables de entrada
├── outputs.tf           # Valores de salida
├── terraform.tfvars    # Valores de variables
└── scripts/
    ├── jenkins_app.sh  # Bootstrap script para Jenkins
    ├── docker_app.sh   # Bootstrap script para Apps (futuro)
    └── ...
```

Archivos Clave

aws_compute.tf - Infraestructura de Compute

Define:

- **aws_instance: jenkins_aws** → VM de Jenkins con user_data
- **aws_instance: app_aws** → VM de aplicaciones (futuro)
- **aws_security_group** → Reglas de firewall (puertos 8080, 22, 50000, etc.)
- **aws_key_pair** → Gestión de claves SSH

Características:

- **user_data** automatiza la instalación de dependencias
- **user_data_replace_on_change** asegura re-ejecución en cambios
- Security groups configuran acceso controlado

scripts/jenkins_app.sh - Bootstrap de Jenkins

Automatiza:

1. Actualización del SO (**dnf update**)
2. Instalación de dependencias (Docker, Git, cURL)
3. Habilitación de servicio Docker
4. Instalación del plugin Docker Compose
5. Generación del archivo **docker-compose.yaml** para Jenkins
6. Levantamiento del contenedor Jenkins
7. **Healthcheck**: Verificación de que Jenkins esté "healthy"
8. Logging en **/var/log/user-data.log**

terraform.tfvars - Variables de Entorno

```
// Ejemplo
aws_region = "eu-west-3"
ssh_public_key = "ssh-rsa AAAAB3..."
```

Proceso de Despliegue

Fase 1: Inicialización (Laboratorios)

```
cd terraform
terraform init
terraform plan
terraform apply
```

Resultado:

- EC2 de Jenkins levantada con Docker + Jenkins running
- Security groups configurados
- SSH key pair operativo

Fase 2: Validación y Testing

- Acceder a Jenkins en `http://<public-ip>:8080`
- Verificar logs: `ssh ec2-user@<ip> → cat /var/log/user-data.log`
- Confirmar healthcheck: `cat /opt/jenkins/.user_data_ran`

Fase 3: Preparación para Producción

Antes de migrar a la cuenta oficial:

1. **Documentar configuraciones** de Jenkins en IaC
2. **Crear Terraform modules** reutilizables
3. **Configurar backend remoto** (S3 + DynamoDB)
4. **Implementar RBAC** y políticas de seguridad
5. **Validar costos** proyectados

Fase 4: Despliegue en Producción

```
# Cambiar credenciales a cuenta oficial AWS
export AWS_PROFILE=production

# Aplicar con variables de producción
terraform apply -var-file=production.tfvars
```

Power Apps: Gestion Simplificada

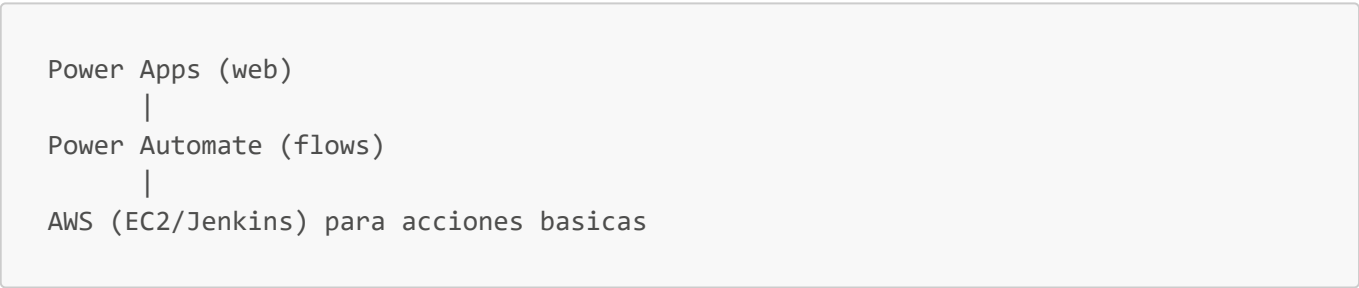
Concepto

La gestion operativa se realizara solo con **Microsoft Power Apps** para tareas sencillas sobre instancias y contenedores.

Funcionalidades Planificadas

Funcionalidad	Descripción	Prioridad
Ver contenedores activos	Listar todos los contenedores running en las VMs	Alta
Relanzar contenedor	<code>docker restart <container></code> desde interfaz	Alta
Detener contenedor	Pausar sin eliminar	Alta
Iniciar contenedor	Arrancar detenido	Alta
Eliminar contenedor	Limpieza de recursos	Media
Ver logs	Acceder a logs de contenedores	Alta
Métricas de recursos	CPU, memoria, disco	Media
Escalado	Fuera de alcance inicial	Baja

Arquitectura



Implementacion (Roadmap)

- 1. **Fase 1:** Dashboard de estado (instancias y contenedores)
- 2. **Fase 2:** Start/Stop/Restart desde Power Apps
- 3. **Fase 3:** Logs basicos y auditoria simple

Security Considerations

Networking

- **VPC:** Usar VPC por defecto (o crear nueva segmentada)
- **Security Groups:**
 - Jenkins: 8080 (web), 50000 (agentes), 22 (SSH)
 - Apps: 22 (SSH), puertos aplicación
- **NACLs:** Restricción de CIDR (no open 0.0.0.0/0 en prod)

Credenciales

- **SSH Keys:** Generadas con `aws_key_pair` en Terraform

- **AWS Credentials:** Usar roles IAM, no access keys directo
- **Secrets:** Usar AWS Secrets Manager (futura integración)

Patching

- **AMI Updates:** Usar datos de AMI "most_recent" (ya implementado)
- **Container Updates:** Jenkins, aplicaciones → vulnerabilidades

Monitoreo y Observabilidad

Logs

- **EC2 user-data:** `/var/log/user-data.log` (ya implementado)
- **Jenkins:** Accesible en UI y via Docker logs
- **CloudWatch:** Centralizar logs de AWS

Métricas

- **CloudWatch Metrics:** CPU, disco, red (gratis básico)
- **Future:** Prometheus en VM App

Alertas

- CloudWatch Alarms (tráfico, CPU, disc space)
- SNS notifications

Costos Estimados (Laboratorios)

Componente	Coste/mes	Notas
EC2 t3.medium (2x)	~\$30	Full uptime
Data transfer	~\$5	Salida data
Storage (EBS 8GB)	~\$1	Minimo
Power Apps (per-user)	~\$10/user	5 operadores = \$50
Power Automate flows	~\$10	Premium connectors
SharePoint/Azure AD	Incluido	M365 existente
Total estimado	~\$106	Incluye gestion operativa basica

Nota: Usar free tier si aplica. En produccion, anadir RDS, load balancers, etc.

Nota: Si tienes Microsoft 365 E3/E5, Power Apps suma menor costo.

Próximos Pasos

Inmediatos (Semana 1-2)

- ☐ Validar despliegue en laboratorios
- ☐ Implementar segundo script ([docker_app.sh](#))
- ☐ Crear VM de aplicaciones
- ☐ Documentar configuraciones de Jenkins

A Corto Plazo (Mes 1)

- ☐ Setup de monitoring (CloudWatch basics)
- ☐ Backup scripts y disaster recovery
- ☐ Validación de seguridad (security audit)

A Medio Plazo (Mes 2-3)

- ☐ Implementación de Power Apps (estado + acciones básicas)
- ☐ CI/CD pipeline operacional
- ☐ Preparación migración a producción

A Largo Plazo (Mes 4+)

- ☐ Migración a cuenta oficial
- ☐ Dashboard web en Power Apps
- ☐ Integración con APM (Prometheus/Grafana)
- ☐ Auto-scaling y orchestration avanzada

Referencias y Recursos

- [Terraform AWS Provider](#)
- [Amazon Linux 2023 AMI](#)
- [Jenkins Docker Hub](#)
- [Docker Compose Documentation](#)

Contacto y Soporte

- **Repositorio:** [Sniifeerns/securenet](#)
- **Rama activa:** [feature/terraform](#)
- **Documentación adicional:** Ver carpeta [/docs](#)